BST & Tree Traversals

The Complete Deep-Dive Guide

Insertion · Deletion · Search · Traversals · Recursion Explained Step-by-Step

BST Operations	4 Traversals	Recursion Explained

Table of Contents

1. Binary Search Tree — The Big Picture

Why BST exists and what makes it special

1.1 From Binary Tree to BST — The One Rule That Changes Everything

A Binary Tree is just any tree where each node has at most 2 children. It has NO ordering rule. So if you want to find a value, you might have to check every single node — O(n) in the worst case.

A Binary Search Tree adds ONE powerful rule:

> **The BST Rule:**
>
> LEFT subtree → all values are SMALLER than the parent node
>
> RIGHT subtree → all values are LARGER than the parent node
>
> This applies recursively to EVERY node in the tree — not just the root!

```
Binary Tree (no rule):          Binary SEARCH Tree (BST):
      [5]                             [20]
     /   \                           /    \
   [8]   [2]                       [15]   [30]
   /       \                       /  \      \
 [1]       [9]                   [12] [18]    [40]

To find 9 in Binary Tree:       To find 18 in BST:
Check 5 → not it                Check 20 → 18 < 20 → go LEFT
Check 8 → not it                Check 15 → 18 > 15 → go RIGHT
Check 2 → not it                Check 18 → FOUND! ✓
Check 1 → not it
Check 9 → FOUND! (5 checks)     Only 3 checks vs up to n checks!

Binary Tree search: O(n)         BST search: O(log n) ✓
```

1.2 Height and Time Complexity — The Core Insight

The number of comparisons needed to find any element = the HEIGHT of the tree. And the height of a balanced BST with n nodes = $\log_2$ (n).

```
n=7 nodes, balanced BST:          Height = log₂(7) ≈ 3
           [20]                   Level 0: 1 node
          /    \                  Level 1: 2 nodes
        [10]   [30]               Level 2: 4 nodes
       /  \    /  \
     [5][15][25][40]
```

```
Worst case: 3 comparisons to find ANY element in 7-node tree!
Compare to linear search: up to 7 comparisons.
For n=1,000,000 nodes: BST needs only log₂(1M) ≈ 20 comparisons!

n=1        → height=0  → 1 comparison max
n=3        → height=1  → 2 comparisons max
n=7        → height=2  → 3 comparisons max
n=15       → height=3  → 4 comparisons max
n=1000000  → height=20 → 20 comparisons max  ← incredibly efficient!
```

Metric	Binary Tree	Binary Search Tree (balanced)	Why it matters
Search	O(n)	O(log n)	BST is exponentially faster for large datasets
Insert	O(1) at known pos	O(log n)	BST must find the right spot first
Delete	O(n)	O(log n)	BST can navigate directly to node
Sorted output	O(n log n) sort needed	O(n) inorder	BST gives sorted data FREE

Interview Trap — 3 Questions That Are The Same!

Q1: Print all elements of a BST in increasing order

Q2: How to check if a tree is a valid BST

Q3: Perform inorder traversal of a BST

Answer: ALL THREE reduce to INORDER TRAVERSAL (Left → Root → Right).

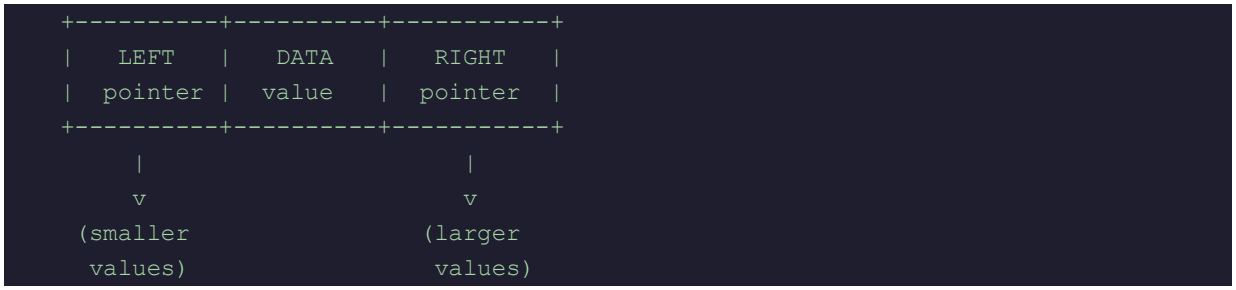
Interviewers phrase it differently to confuse you. Now you know!

🌳 2. The Node — Building Block of BST

What each node stores and why

2.1 Node Structure

Every BST is made of nodes. Each node stores three things: the data value, a pointer to the left child, and a pointer to the right child.



Python Implementation

```
class Node:
    def __init__(self, value):
        # The actual data this node holds
        self.data  = value

        # Pointer to LEFT child (smaller values live here)
        # None means 'no left child' (this node is a leaf on left side)
        self.left  = None

        # Pointer to RIGHT child (larger values live here)
        # None means 'no right child'
        self.right = None

# Time:  O(1) — just initializing 3 attributes
# Space: O(1) — only 3 fields stored per node
```

C / C++ Implementation

```
// C struct approach
struct Node {
    int data;          // value stored
    Node* left;        // pointer to left child
    Node* right;       // pointer to right child
};

// C++ class approach (constructor)
class Node {
```

```
    public:
        int data;
        Node* left;
        Node* right;

        Node(int value) {
            data  = value;
            left  = nullptr;   // nullptr = no child
            right = nullptr;
        }
    };
    // Time: O(1),  Space: O(1)
```

✚ 3. Insertion in BST

How values are placed — the recursive journey

3.1 The Insertion Logic

To insert a value into a BST, we follow the BST rule at every node:

- If the tree is empty → the new node becomes the root
- If value < current node → go LEFT
- If value > current node → go RIGHT
- If value == current node → duplicate, ignore it (BSTs usually don't store duplicates)
- Keep going until we find an empty (None/null) spot → insert there

3.2 Step-by-Step: Building the BST

Let's insert: 20, 15, 30, 12, 18, 40 — one by one.

```
Step 1: Insert 20
Tree is empty → 20 becomes root
[20]

Step 2: Insert 15
15 < 20 → go LEFT of 20
20's left is None → place 15 there
     [20]
     /
   [15]

Step 3: Insert 30
30 > 20 → go RIGHT of 20
20's right is None → place 30 there
     [20]
     /  \
   [15]  [30]

Step 4: Insert 12
12 < 20 → go LEFT  (reach 15)
12 < 15 → go LEFT  (reach None)
Place 12 at 15's left
       [20]
       /  \
     [15]  [30]
     /
   [12]

Step 5: Insert 18
```

```
18 < 20 → go LEFT  (reach 15)
18 > 15 → go RIGHT (reach None)
Place 18 at 15's right
       [20]
       /  \
     [15]  [30]
     /  \
   [12] [18]

Step 6: Insert 40
40 > 20 → go RIGHT (reach 30)
40 > 30 → go RIGHT (reach None)
Place 40 at 30's right
       [20]
       /  \
     [15]  [30]
     /  \     \
   [12] [18]  [40]    ← FINAL TREE
```

3.3 Recursive Code — Explained Line by Line

Python

```
def insertion_in_bst(root, value):
    # ─── BASE CASE ──────────────────────────────────────────────
    # We've reached an empty spot in the tree (None node).
    # This is exactly where the new value belongs!
    # Create a new Node here and return it so the parent can
    # connect to it via root.left = ... or root.right = ...
    #
    # Time: O(1) for this operation
    # Space: O(1) for the new node created
    if root is None:
        return Node(value)          # new node born here

    # ─── DUPLICATE CHECK ────────────────────────────────────────
    # BSTs typically don't allow duplicates.
    # If value already exists, just return root unchanged.
    elif value == root.data:
        return root                 # ignore duplicate

    # ─── GO LEFT ────────────────────────────────────────────────
    # value is smaller → it belongs in the LEFT subtree.
    # We recursively insert into the left subtree and
    # REASSIGN root.left to the result.
    # Why reassign? Because the recursive call might return a
    # brand new Node (when root.left was None) — we must link it!
    elif value < root.data:
        root.left = insertion_in_bst(root.left, value)
```

```
        # ─── GO RIGHT ──────────────────────────────────────────────
        # value is larger → it belongs in the RIGHT subtree.
        # Same logic as left, but for right side.
        else:
            root.right = insertion_in_bst(root.right, value)

        # ─── RETURN ROOT ───────────────────────────────────────────
        # CRITICAL: always return root so the parent call can
        # reconnect the subtree correctly.
        # Without this, the tree gets disconnected!
        return root

    # Time Complexity:
    #   Best:    O(1)      — tree is empty, just create root
    #   Average: O(log n)  — balanced tree, traverse half height
    #   Worst:   O(n)      — skewed tree (sorted input), traverse all nodes
    # Space Complexity:
    #   O(h) — h = height of tree (recursion call stack frames)
    #   Balanced: O(log n)   Skewed: O(n)
```

C++ Implementation

```
    Node* insertBST(Node* root, int value) {
        // Base case: empty spot found → create new node here
        // nullptr in C++ = None in Python
        if (root == nullptr)
            return new Node(value);

        // Duplicate: ignore
        if (value == root->data)
            return root;

        // Go LEFT: value smaller than current node
        // root->left uses -> because root is a POINTER
        if (value < root->data)
            root->left = insertBST(root->left, value);

        // Go RIGHT: value larger than current node
        else
            root->right = insertBST(root->right, value);

        // Return root to maintain parent linkage
        return root;
    }

    // Usage:
    // Node* root = nullptr;
    // root = insertBST(root, 20);
    // root = insertBST(root, 15);
    // root = insertBST(root, 30);
```

3.4 Recursion Call Stack — Full Trace for insert(20→15→30→12)

Let's trace EXACTLY what happens in memory when we insert 12 into the tree that already has 20, 15, 30:

Case	Scenario	Time	Space	Example
Best	Tree is empty — new node becomes root	O(1)	O(1)	Insert into empty tree
Average	Balanced BST — traverse ~log(n) nodes	O(log n)	O(log n)	Insert 25 into balanced 7-node tree
Worst	Skewed tree — traverse all n nodes	O(n)	O(n)	Insert 1,2,3,4,5 in order → right skewed

4. Search in BST

The power of O(log n) — finding elements fast

4.1 How Search Works

Searching in a BST is like playing a 'higher or lower' guessing game. At each node, you know exactly which direction to go — left (if target is smaller) or right (if target is larger). This eliminates half the remaining tree at every step.

```
BST for search examples:
           [20]
          /    \
        [15]   [30]
        /  \      \
      [12] [18]   [40]

Search for 18:
Step 1: Compare 18 with root 20  → 18 < 20 → go LEFT
Step 2: Compare 18 with node 15  → 18 > 15 → go RIGHT
Step 3: Compare 18 with node 18  → 18 == 18 → FOUND! ✓
Only 3 comparisons for 6-node tree!

Search for 25 (not in tree):
Step 1: Compare 25 with root 20  → 25 > 20 → go RIGHT
Step 2: Compare 25 with node 30  → 25 < 30 → go LEFT
Step 3: 30's left is None        → NOT FOUND
3 comparisons to confirm 25 is absent!
```

4.2 Search Code — Every Line Explained

Python

```
def search_in_bst(root, value):
    # ─── BASE CASE 1: Not found ──────────────────────────────────
    # We've gone past a leaf node (reached None/null).
    # The value doesn't exist anywhere in this subtree.
    # This happens when we kept going left or right and fell off the tree.
    if root is None:
        print(f'Element {value} not found')
        return False

    # ─── BASE CASE 2: Found! ─────────────────────────────────────
    # Current node's data matches what we're looking for.
    # No need to go further — report success.
    elif value == root.data:
        print(f'Element {value} found')
```

```
            return True

        # ─── RECURSIVE CASE: Go LEFT ──────────────────────────────
        # Target value is SMALLER than current node.
        # By BST property, it can ONLY exist in the left subtree.
        # Right subtree guaranteed to not have it → skip entirely!
        elif value < root.data:
            return search_in_bst(root.left, value)

        # ─── RECURSIVE CASE: Go RIGHT ─────────────────────────────
        # Target value is LARGER than current node.
        # By BST property, it can ONLY exist in the right subtree.
        else:
            return search_in_bst(root.right, value)

    # Time Complexity:
    #   Best:    O(1)      — value is at root
    #   Average: O(log n)  — balanced BST, cut tree in half each step
    #   Worst:   O(n)      — skewed tree OR value at deepest leaf
    # Space Complexity:
    #   O(h) due to recursion stack (h = height of tree)
    #   Balanced: O(log n)    Skewed: O(n)
```

C++

```
    bool searchBST(Node* root, int value) {
        // Base case 1: fell off the tree → not found
        if (root == nullptr)
            return false;

        // Base case 2: exact match → found!
        if (value == root->data)
            return true;

        // Recursive: search left or right based on comparison
        if (value < root->data)
            return searchBST(root->left, value);
        else
            return searchBST(root->right, value);
    }
```

4.3 Recursion Trace for search(root=20, value=18)

```
    Call Stack builds up (top = most recent call):


    ┌──────────────────────────────────────────────────────────┐
    │ search(root=Node(18), value=18)                          │
    │ 18 == 18 → BASE CASE 2: return True ← unwinds here       │
    ├──────────────────────────────────────────────────────────┤
```

```
    | search(root=Node(15), value=18)                              |
    | 18 > 15 → go RIGHT → call search(Node(18), 18)               |
    | returns True ← gets result, passes it up                     |
    ├──────────────────────────────────────────────────────────────┤
    | search(root=Node(20), value=18)  ← FIRST call                |
    | 18 < 20 → go LEFT  → call search(Node(15), 18)               |
    | returns True ← final answer bubbles up to caller             |
    └──────────────────────────────────────────────────────────────┘


   Each frame is pushed when we recurse, popped when we return.
   Memory freed for each frame as soon as it returns.
```

5. Deletion in BST

The hardest operation — 3 cases, explained clearly

5.1 Predecessor and Successor — Must Know First

Before deletion, understand two key concepts:

> **In-Order Predecessor and Successor:**
>
> Given a node N in a BST after inorder traversal [6,8,9,10,20,25,30,40,50]:
>
> In-Order Predecessor of N = the element just BEFORE N in sorted order
>
> In-Order Successor of N = the element just AFTER N in sorted order
>
> For N=25: Predecessor=20 (one step LEFT then rightmost),
>
> Successor=30 (one step RIGHT then leftmost)
>
> For N=10: Predecessor=9, Successor=20

```
How to find In-Order SUCCESSOR of node N:
1. Go ONE step to the RIGHT child of N
2. Then go as FAR LEFT as possible
3. The leftmost node = smallest value larger than N = Successor

Example: Successor of 25 in tree below:
           [20]
          /    \
        [10]   [30]
        /  \   /  \
      [6] [15][25] [40]
               \
               [28]

Step 1: from 25, go RIGHT → reach 28... wait, 25's right = 28
Actually 25 has no right child shown, go to parent's right subtree...

Cleaner example — Successor of 10:
Step 1: Go right from 10 → reach 15
Step 2: Go as far left as possible from 15 → 15 has no left child
Result: Successor of 10 = 15 ✓

How to find In-Order PREDECESSOR of node N:
1. Go ONE step to the LEFT child of N
```

```
    2. Then go as FAR RIGHT as possible
    3. The rightmost node = largest value smaller than N = Predecessor
```

5.2 The 3 Cases of Deletion

When deleting a node from a BST, there are exactly 3 situations. Each is handled differently.

Case 1: Deleting a LEAF Node (no children)

```
    Delete node 12 (a leaf):          After deletion:
           [20]                              [20]
           /  \                              /  \
        [15]  [30]           →            [15]  [30]
        /  \     \                           \     \
     [12] [18]  [40]                        [18]  [40]

    Action: Simply return None to the parent.
    Parent (15) had root.left = Node(12).
    After deletion, root.left = None.
    Memory for Node(12) is freed.

    Code handles it automatically: if root.left is None → return root.right
    When BOTH are None (leaf), returns None → parent disconnects it.
```

Case 2: Deleting a Node with ONE child

```
    Delete node 30 (has only right child 40):
           [20]                              [20]
           /  \                              /  \
        [15]  [30]           →            [15]  [40]
        /  \     \                         /  \
     [12] [18]  [40]                   [12] [18]

    Action: Connect 30's parent directly to 30's child.
    Parent (20) had root.right = Node(30).
    After deletion, root.right = Node(40). (bypasses 30)
    Node(30) is removed. Node(40) takes its place.

    Code: if root.left is None → return root.right  (right child takes over)
          if root.right is None → return root.left  (left child takes over)
```

Case 3: Deleting a Node with TWO children (the tricky one!)

```
    Delete node 20 (has both left=15 and right=30):

    PROBLEM: Can't just remove it — two subtrees need a new root!

    SOLUTION: Replace with IN-ORDER SUCCESSOR (or predecessor)
```

```
Why successor? Because it is:
- The smallest value LARGER than 20
- Guaranteed to be > everything in left subtree (maintains BST!)
- Guaranteed to be < everything in right subtree (maintains BST!)

Full BST for this example:
           [20]                          [25]
          /    \                        /    \
        [15]   [30]      →            [15]   [30]
        /  \   /  \                   /  \      \
      [12][18][25] [40]             [12][18]    [40]

Steps:
1. Find inorder successor of 20:
   Go RIGHT to 30, then go as FAR LEFT as possible → reach 25
   Successor = 25

2. COPY successor's value (25) into the node being deleted (20)
   Tree now has two 25s: one at old root, one at original position
   [6,8,9,10,15,18,25,25,30,40,50] — still valid BST!

3. DELETE the original successor node (25) from right subtree
   25 has no left child → Case 2 applies (easy delete)

Result: Node that had 20 now has 25, and old 25 is gone. ✓
```

5.3 In-Order Successor Helper Function

Python

```python
def get_in_order_successor(root):
    """
    Find the in-order successor of a node.
    Strategy: go one step RIGHT, then as far LEFT as possible.
    Result: smallest value in the right subtree = in-order successor.
    """
    # Step 1: Move to the right child of the node to be deleted
    # This right child is larger than the deleted node
    root = root.right

    # Step 2: Keep going left until we can't anymore
    # The leftmost node of the right subtree = smallest value
    # that is still larger than the deleted node
    while root is not None and root.left is not None:
        root = root.left

    # Return the successor node (not just its value, the whole node)
    return root
```

```
    # Example trace for finding successor of 20 in tree above:
    # root.right = Node(30)   ← step right
    # Node(30).left = Node(25) ← step left
    # Node(25).left = None    ← stop, leftmost reached
    # Return Node(25)  → 25 is the successor of 20 ✓

    # Time:  O(h) — traverse from right child to leftmost leaf
    # Space: O(1) — just moves a pointer, no recursion
```

5.4 Delete Function — Full Code with Every Line Explained

Python

```
def delete_from_bst(root, value):
    # ─── BASE CASE: Value not in tree ──────────────────────────────
    # Reached None → value was never found → nothing to delete
    if root is None:
        print(f'Element {value} not found')
        return None

    # ─── NAVIGATE LEFT ─────────────────────────────────────────────
    # Value is smaller → the node to delete is in the left subtree
    # Recurse left and REASSIGN to update the link correctly
    elif value < root.data:
        root.left = delete_from_bst(root.left, value)

    # ─── NAVIGATE RIGHT ────────────────────────────────────────────
    # Value is larger → the node to delete is in the right subtree
    elif value > root.data:
        root.right = delete_from_bst(root.right, value)

    # ─── FOUND THE NODE TO DELETE ──────────────────────────────────
    # root.data == value → this is our target!
    else:
        # Case 1 & 2: Only right child (or no children at all)
        # Return root.right → parent will link to it directly
        # If right is also None (leaf), returns None → parent disconnects
        if root.left is None:
            return root.right

        # Case 2 (mirror): Only left child
        # Return root.left → parent links to left child
        elif root.right is None:
            return root.left

        # Case 3: Two children — the complex case
        # Cannot remove directly: need to maintain BST structure
        # Strategy: find inorder successor, copy its value here,
```

```
            # then delete the original successor from right subtree
            succ = get_in_order_successor(root)
            # Copy successor's value into this node
            root.data = succ.data
            # Delete successor from right subtree
            # Successor has at most ONE child (no left child by definition!)
            # So this deletion will fall into Case 1 or Case 2
            root.right = delete_from_bst(root.right, succ.data)

        # ─── RETURN ROOT ──────────────────────────────────────────
        # Always return root so parent links stay intact
        return root

    # Time Complexity:
    #   Best:    O(1)       — root is the node to delete, it's a leaf
    #   Average: O(log n)   — balanced BST
    #   Worst:   O(n)       — skewed tree
    # Space Complexity: O(h) — recursion call stack
```

C++

```
    Node* getSuccessor(Node* root) {
        root = root->right;               // one step right
        while (root && root->left)         // go as far left as possible
            root = root->left;
        return root;
    }

    Node* deleteBST(Node* root, int value) {
        if (root == nullptr) return nullptr;  // not found

        if (value < root->data)
            root->left = deleteBST(root->left, value);
        else if (value > root->data)
            root->right = deleteBST(root->right, value);
        else {
            // Case 1 & 2: 0 or 1 child
            if (root->left == nullptr) {
                Node* temp = root->right;
                delete root;          // free memory in C++!
                return temp;
            }
            if (root->right == nullptr) {
                Node* temp = root->left;
                delete root;
                return temp;
            }
            // Case 3: two children
            Node* succ = getSuccessor(root);
            root->data = succ->data;
```

```
            root->right = deleteBST(root->right, succ->data);
        }
        return root;
    }
    // Note: In C++, always 'delete' the node to free heap memory!
    // Python handles this automatically (garbage collection).
```

5.5 Deletion Recursion Trace — Deleting 12 (Leaf)

```
Tree before: [20]->[15]->[12] (left), [18] (right), [30]->[40]
Call: delete_from_bst(root=Node(20), value=12)


┌──────────────────────────────────────────────────────────────┐
│ Frame 1: root=Node(20), value=12                             │
│ 12 < 20 → go LEFT                                            │
│ root.left = delete_from_bst(Node(15), 12)  ← RECURSE         │
│ (waiting...)                                                 │
└──────────────────────────────────────────────────────────────┘
        ↓
┌──────────────────────────────────────────────────────────────┐
│ Frame 2: root=Node(15), value=12                             │
│ 12 < 15 → go LEFT                                            │
│ root.left = delete_from_bst(Node(12), 12)  ← RECURSE         │
│ (waiting...)                                                 │
└──────────────────────────────────────────────────────────────┘
        ↓
┌──────────────────────────────────────────────────────────────┐
│ Frame 3: root=Node(12), value=12                             │
│ 12 == 12 → FOUND! Target node!                               │
│ root.left is None AND root.right is None → LEAF              │
│ if root.left is None → return root.right = None              │
└──────────────────────────────────────────────────────────────┘
        ↑ returns None
┌──────────────────────────────────────────────────────────────┐
│ Frame 2 resumes: root.left = None                            │
│ Node(15).left is now None (12 is gone) ✓                     │
│ return Node(15)                                              │
└──────────────────────────────────────────────────────────────┘
        ↑ returns Node(15)
┌──────────────────────────────────────────────────────────────┐
│ Frame 1 resumes: root.left = Node(15) (unchanged, relinked)  │
│ return Node(20)                                              │
└──────────────────────────────────────────────────────────────┘


Tree after: [20]->[15]->[18] (right only), [30]->[40]
Node 12 is gone. Tree is still a valid BST. ✓
```

Case	Scenario	Action	Time	Space
Case 1	Leaf node (0 children)	Return None to parent. Parent disconnects it.	O(log n) avg	O(h)
Case 2	1 child	Return the child to parent. Child takes deleted node's place.	O(log n) avg	O(h)
Case 3	2 children	Find successor, copy value, delete successor (Case 1 or 2).	O(log n) avg	O(h)

6. Tree Traversals — Why We Need Them

A tree isn't linear — so how do we visit every node?

6.1 The Traversal Problem

In an array or linked list, visiting every element is trivial — just go from left to right. But a tree branches! From node 1, you can go to node 2 OR node 3. Which do you visit first? This is the traversal question.

```
Simple tree:      [1]
                 /   \
               [2]   [3]

All possible orderings of {1, 2, 3}:
1 2 3  |  1 3 2  |  2 1 3  |  2 3 1  |  3 1 2  |  3 2 1
(that is 3! = 6 possible orders)

We focus on ROOT position, always going LEFT before RIGHT:

Category 1 — Root at START (Preorder):
  Root → Left → Right  →  1 2 3

Category 2 — Root in MIDDLE (Inorder):
  Left → Root → Right  →  2 1 3

Category 3 — Root at END (Postorder):
  Left → Right → Root  →  2 3 1

3 standard traversals selected (one from each category).
(We skip Root→Right→Left etc. by convention: always left before right)
```

Why These 3 Specifically?

By convention: we ALWAYS visit the LEFT child before the RIGHT child.

This halves the choices: 6 orderings → 3 standard traversals.

Each has a specific use case — not arbitrary choices!

1□ 7. Pre-Order Traversal

Root → Left → Right

7.1 The Pattern

Visit the ROOT first, then recursively traverse the LEFT subtree, then the RIGHT subtree. You see the parent BEFORE its children.

```
Tree:                Preorder Result:
      [1]
     /   \           Visit root  →  print 1
   [3]   [5]         Go left     →  print 3
   /  \    \         3's left    →  print 2
 [2]  [4]  [8]       3's right   →  print 4
                      Go right    →  print 5
                      5's right   →  print 8

Preorder: 1 → 3 → 2 → 4 → 5 → 8
```

7.2 Code — Line by Line

Python

```python
def pre_order_traversal(root):
    # ─── BASE CASE ──────────────────────────────────────────────
    # Empty node → nothing to visit → return empty list
    # This triggers when we try to go left/right from a leaf
    #
    # Time: O(1) for this check
    if root is None:
        return []

    # ─── VISIT ROOT FIRST (Pre = before children) ──────────────
    # Put current node's data in result list BEFORE recursing
    # This is what makes it 'pre'-order
    #
    # Then recurse left: get all data from left subtree as a list
    # Then recurse right: get all data from right subtree as a list
    # Concatenate: [root] + [left subtree] + [right subtree]
    return (
        [root.data]                      +  # root first
        pre_order_traversal(root.left)   +  # then left subtree
        pre_order_traversal(root.right)     # then right subtree
    )

# Time Complexity:
```

```
    #   Best/Average/Worst: O(n) — must visit every single node
    # Space Complexity:
    #   O(h) for call stack + O(n) for the result list
    #   Balanced: O(log n) stack depth   Skewed: O(n) stack depth
```

C++

```
    void preOrder(Node* root, vector<int>& result) {
        if (root == nullptr) return;  // base case

        result.push_back(root->data);  // visit root FIRST
        preOrder(root->left, result);  // then left subtree
        preOrder(root->right, result); // then right subtree
    }

    // Alternative: return vector (cleaner but slightly less efficient)
    vector<int> preOrder(Node* root) {
        if (!root) return {};
        vector<int> res = {root->data};
        auto left  = preOrder(root->left);
        auto right = preOrder(root->right);
        res.insert(res.end(), left.begin(),  left.end());
        res.insert(res.end(), right.begin(), right.end());
        return res;
    }
```

7.3 Full Recursion Trace — Step by Step

Tree: Node(1), left=Node(3)[left=Node(2), right=Node(4)], right=Node(5)[right=Node(8)]

```
    pre_order_traversal(Node(1))  ← initial call
    ├── root=Node(1), not None
    ├── [1] + pre_order(Node(3)) + pre_order(Node(5))
    │
    ├── pre_order(Node(3)):
    │   ├── [3] + pre_order(Node(2)) + pre_order(Node(4))
    │   │
    │   ├── pre_order(Node(2)):
    │   │   ├── [2] + pre_order(None) + pre_order(None)
    │   │   ├── pre_order(None) → []   ← base case
    │   │   ├── pre_order(None) → []   ← base case
    │   │   └── returns [2]
    │   │
    │   ├── pre_order(Node(4)):
    │   │   ├── [4] + pre_order(None) + pre_order(None)
    │   │   └── returns [4]
    │   │
    │   └── returns [3] + [2] + [4] = [3, 2, 4]
    │
```

```
    ├── pre_order(Node(5)):
    │   ├── [5] + pre_order(None) + pre_order(Node(8))
    │   │
    │   ├── pre_order(Node(8)):
    │   │   └── returns [8]
    │   │
    │   └── returns [5] + [] + [8] = [5, 8]
    │
    └── Final: [1] + [3,2,4] + [5,8] = [1, 3, 2, 4, 5, 8]  ✓
```

Preorder Use Cases:

Copying/cloning a tree: insert nodes in preorder into new tree = exact copy

Serializing to file: store preorder sequence → rebuild identical tree later

Prefix expression evaluation: math like + * 3 4 2 uses preorder notation

Directory listing: print folder name before its contents (ls -R style)

2□ 8. In-Order Traversal

Left → Root → Right = Sorted Output!

8.1 The Pattern + Secret Superpower

Visit the LEFT subtree first, then the ROOT, then the RIGHT subtree. For a BST, this ALWAYS produces output in sorted ascending order. This is the most important traversal for BSTs.

```
BST:                  Inorder Result:
      [1]
     /   \            Go far left   →  visit 2
   [3]   [5]          Back to 3     →  visit 3
   /  \    \          3's right     →  visit 4
 [2]  [4]  [8]        Back to root  →  visit 1
                       Go right      →  visit 5
                       5's right     →  visit 8

Inorder: 2 → 3 → 4 → 1 → 5 → 8

For a BST with values [12,15,18,20,30,40]:
Inorder always gives: 12 → 15 → 18 → 20 → 30 → 40  (sorted!) ✓
```

8.2 Code — Line by Line

Python

```python
def in_order_traversal(root):
    # ─── BASE CASE ──────────────────────────────────────────────
    # Reached a None node (went past a leaf).
    # Nothing to visit here → return empty list.
    if root is None:
        return []

    # ─── LEFT → ROOT → RIGHT ────────────────────────────────────
    # 1) FIRST recurse into left subtree (get all smaller values)
    # 2) THEN add current node's data (the 'root' in L-R-R)
    # 3) THEN recurse into right subtree (get all larger values)
    #
    # Result: left_values + [current] + right_values
    # In a BST: this is always in ASCENDING ORDER!
    return (
        in_order_traversal(root.left)    +  # all smaller values first
        [root.data]                      +  # then this node
        in_order_traversal(root.right)      # then all larger values
    )
```

```
# Time Complexity: O(n) — must visit every node exactly once
# Space Complexity:
#   Call stack: O(h) — h = height (O(log n) balanced, O(n) skewed)
#   Result list: O(n) — stores all n values
# KEY PROPERTY: On a BST → always gives SORTED output!
```

C++

```
void inOrder(Node* root, vector<int>& result) {
    if (root == nullptr) return;

    inOrder(root->left, result);       // left FIRST
    result.push_back(root->data);      // then root
    inOrder(root->right, result);      // then right
}
```

8.3 Full Recursion Trace

```
in_order_traversal(Node(1)):
├── in_order(Node(3)) + [1] + in_order(Node(5))
│
├── in_order(Node(3)):
│   ├── in_order(Node(2)) + [3] + in_order(Node(4))
│   │
│   ├── in_order(Node(2)):
│   │   ├── in_order(None) → []
│   │   ├── [2]
│   │   ├── in_order(None) → []
│   │   └── returns [] + [2] + [] = [2]
│   │
│   ├── in_order(Node(4)):
│   │   └── returns [4]
│   │
│   └── returns [2] + [3] + [4] = [2, 3, 4]
│
├── [1]
│
├── in_order(Node(5)):
│   ├── in_order(None) → []
│   ├── [5]
│   ├── in_order(Node(8)) → [8]
│   └── returns [5, 8]
│
└── Final: [2,3,4] + [1] + [5,8] = [2, 3, 4, 1, 5, 8]

NOTE: This tree is NOT a BST (values aren't in BST order),
so inorder doesn't give perfect sort here.
On a proper BST (12,15,18,20,30,40), inorder = [12,15,18,20,30,40] ✓
```

3 9. Post-Order Traversal

Left → Right → Root

9.1 The Pattern

Visit LEFT subtree first, then RIGHT subtree, then the ROOT last. Children are always processed before their parent. This is the 'clean up after children' traversal.

```
Tree:                  Postorder Result:
       [1]
      /   \            Go to leaves first:
    [3]   [5]          visit 2  (leftmost leaf)
    /  \    \          visit 4  (right leaf of 3)
  [2]  [4]  [8]        visit 3  (after both children done)
                        visit 8  (only child of 5)
                        visit 5  (after child done)
                        visit 1  (root — always LAST)

Postorder: 2 → 4 → 3 → 8 → 5 → 1
Notice: Root (1) is ALWAYS last in postorder!
```

9.2 Code — Line by Line

Python

```python
def post_order_traversal(root):
    # ── BASE CASE ──────────────────────────────────────────────
    # Reached a None node → nothing here → return empty list
    if root is None:
        return []

    # ── LEFT → RIGHT → ROOT ────────────────────────────────────
    # 1) FIRST recurse into left subtree completely
    # 2) THEN  recurse into right subtree completely
    # 3) LAST  add the current root's value
    #
    # Why is this useful?
    # Parent is processed AFTER all its descendants.
    # Perfect for: deletion (delete children before parent),
    # dependency resolution (finish deps before task),
    # postfix expression evaluation
    return (
        post_order_traversal(root.left)   +  # left subtree first
        post_order_traversal(root.right)  +  # right subtree second
        [root.data]                          # current node LAST
    )
```

```
# Time:  O(n) — every node visited exactly once
# Space: O(h) call stack + O(n) result list
# KEY: Root is ALWAYS the last element in postorder output
```

C++

```
void postOrder(Node* root, vector<int>& result) {
    if (root == nullptr) return;

    postOrder(root->left, result);     // left first
    postOrder(root->right, result);    // right second
    result.push_back(root->data);      // root LAST
}
```

9.3 Full Recursion Trace

```
post_order_traversal(Node(1)):
├── post_order(Node(3)) + post_order(Node(5)) + [1]
│
├── post_order(Node(3)):
│   ├── post_order(Node(2)) + post_order(Node(4)) + [3]
│   │
│   ├── post_order(Node(2)):
│   │   ├── post_order(None) → []
│   │   ├── post_order(None) → []
│   │   └── returns [] + [] + [2] = [2]
│   │
│   ├── post_order(Node(4)):
│   │   └── returns [4]
│   │
│   └── returns [2] + [4] + [3] = [2, 4, 3]
│
├── post_order(Node(5)):
│   ├── post_order(None) → []
│   ├── post_order(Node(8)) → [8]
│   └── returns [] + [8] + [5] = [8, 5]
│
└── Final: [2,4,3] + [8,5] + [1] = [2, 4, 3, 8, 5, 1]  ✓

Root 1 appears LAST ✓
Both children of any node appear before it ✓
```

> **Postorder Use Cases:**
>
> Tree deletion: always delete children before parent (no orphaned nodes)
>
> Postfix (RPN) expression evaluation: operands before operator

> Folder size calculation: calculate subfolder size before parent folder
>
> Dependency resolution: finish dependencies before the dependent task
>
> Build systems: compile dependencies before the module that uses them

10. Complete Program

All operations together — Python & C++

10.1 Full Python Program

```
# ================================================================
# COMPLETE BST IMPLEMENTATION
# Insertion | Search | Deletion | Traversals
# ================================================================

class Node:
    def __init__(self, value):
        self.data  = value
        self.left  = None
        self.right = None

# ── INSERTION ──────────────────────────────────────────────
# Time: Best O(1), Avg O(log n), Worst O(n) | Space: O(h)
def insertion_in_bst(root, value):
    if root is None:          return Node(value)
    elif value == root.data:  return root
    elif value < root.data:   root.left  = insertion_in_bst(root.left,  value)
    else:                     root.right = insertion_in_bst(root.right, value)
    return root

# ── SEARCH ─────────────────────────────────────────────────
# Time: Best O(1), Avg O(log n), Worst O(n) | Space: O(h)
def search_in_bst(root, value):
    if root is None:          print(f'{value} not found');  return False
    elif value == root.data:  print(f'{value} found');      return True
    elif value < root.data:   return search_in_bst(root.left,  value)
    else:                     return search_in_bst(root.right, value)

# ── INORDER SUCCESSOR HELPER ───────────────────────────────
# Time: O(h) | Space: O(1)
def get_in_order_successor(root):
    root = root.right
    while root is not None and root.left is not None:
        root = root.left
    return root

# ── DELETION ───────────────────────────────────────────────
# Time: Best O(1), Avg O(log n), Worst O(n) | Space: O(h)
def delete_from_bst(root, value):
    if root is None:      print(f'{value} not found'); return None
    elif value < root.data: root.left  = delete_from_bst(root.left,  value)
    elif value > root.data: root.right = delete_from_bst(root.right, value)
```

```
        else:
            if root.left is None:  return root.right  # Case 1/2
            if root.right is None: return root.left   # Case 2
            succ = get_in_order_successor(root)       # Case 3
            root.data  = succ.data
            root.right = delete_from_bst(root.right, succ.data)
        return root

    # ── TRAVERSALS ──────────────────────────────────────────
    # All: Time O(n), Space O(h) call stack + O(n) result
    def pre_order_traversal(root):
        if root is None: return []
        return [root.data] + pre_order_traversal(root.left) +
pre_order_traversal(root.right)

    def in_order_traversal(root):
        if root is None: return []
        return in_order_traversal(root.left) + [root.data] +
in_order_traversal(root.right)

    def post_order_traversal(root):
        if root is None: return []
        return post_order_traversal(root.left) + post_order_traversal(root.right) +
[root.data]

    # ── BUILD BST ───────────────────────────────────────────
    root = None
    for val in [20, 15, 30, 12, 18, 40, 25, 50]:
        root = insertion_in_bst(root, val)

    # ── OUTPUTS ─────────────────────────────────────────────
    print('Inorder  (sorted):', in_order_traversal(root))
    # → [12, 15, 18, 20, 25, 30, 40, 50]
    print('Preorder         :', pre_order_traversal(root))
    # → [20, 15, 12, 18, 30, 25, 40, 50]
    print('Postorder        :', post_order_traversal(root))
    # → [12, 18, 15, 25, 50, 40, 30, 20]

    search_in_bst(root, 18)   # 18 found
    search_in_bst(root, 100)  # 100 not found

    root = delete_from_bst(root, 12)   # delete leaf
    print('After deleting 12:', in_order_traversal(root))
    # → [15, 18, 20, 25, 30, 40, 50]

    root = delete_from_bst(root, 30)   # delete 2-child node
    print('After deleting 30:', in_order_traversal(root))
    # → [15, 18, 20, 25, 40, 50]
```

10.2 Full C++ Program

```
#include <iostream>
#include <vector>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int v) : data(v), left(nullptr), right(nullptr) {}
};

Node* insert(Node* root, int val) {
    if (!root)          return new Node(val);
    if (val < root->data) root->left  = insert(root->left,  val);
    else if (val > root->data) root->right = insert(root->right, val);
    return root;
}

bool search(Node* root, int val) {
    if (!root) return false;
    if (val == root->data) return true;
    return val < root->data ? search(root->left, val) : search(root->right,
val);
}

Node* getSuccessor(Node* root) {
    root = root->right;
    while (root && root->left) root = root->left;
    return root;
}

Node* deleteNode(Node* root, int val) {
    if (!root) return nullptr;
    if (val < root->data)  { root->left  = deleteNode(root->left,  val); }
    else if (val > root->data) { root->right = deleteNode(root->right, val); }
    else {
        if (!root->left)  { Node* t = root->right; delete root; return t; }
        if (!root->right) { Node* t = root->left;  delete root; return t; }
        Node* succ = getSuccessor(root);
        root->data = succ->data;
        root->right = deleteNode(root->right, succ->data);
    }
    return root;
}

void preOrder(Node* root, vector<int>& v) {
    if (!root) return;
    v.push_back(root->data);
    preOrder(root->left, v);
```

```
    preOrder(root->right, v);
}

void inOrder(Node* root, vector<int>& v) {
    if (!root) return;
    inOrder(root->left, v);
    v.push_back(root->data);
    inOrder(root->right, v);
}

void postOrder(Node* root, vector<int>& v) {
    if (!root) return;
    postOrder(root->left, v);
    postOrder(root->right, v);
    v.push_back(root->data);
}

int main() {
    Node* root = nullptr;
    for (int v : {20, 15, 30, 12, 18, 40, 25, 50})
        root = insert(root, v);

    vector<int> res;
    inOrder(root, res);
    // prints: 12 15 18 20 25 30 40 50
    return 0;
}
```

⚡ 11. Master Complexity Reference

Every operation, every case

11.1 BST Operations

Operation	Best Case	Average Case	Worst Case	Space	When worst occurs
Insert	O(1)	O(log n)	O(n)	O(h)	Sorted input → skewed tree
Search	O(1)	O(log n)	O(n)	O(h)	Element at deepest leaf or absent
Delete (leaf)	O(log n)	O(log n)	O(n)	O(h)	Leaf at deepest level of skewed tree
Delete (2 child)	O(log n)	O(log n)	O(n)	O(h)	Skewed tree + node near bottom
Find Min	O(1)	O(log n)	O(n)	O(1)	Leftmost node in skewed-right tree
Find Max	O(1)	O(log n)	O(n)	O(1)	Rightmost node in skewed-left tree
Get Successor	O(1)	O(log n)	O(n)	O(1)	Successor at deepest level

11.2 Traversal Complexities

Traversal	Pattern	Time	Stack Space (balanced)	Stack Space (skewed)	Result Space	Key Use
Preorder	Root→L→R	O(n)	O(log n)	O(n)	O(n)	Copy tree, serialize
Inorder	L→Root→R	O(n)	O(log n)	O(n)	O(n)	Sorted output from BST
Postorder	L→R→Root	O(n)	O(log n)	O(n)	O(n)	Delete tree, postfix eval

11.3 Key Rules to Memorize

- Inorder of a BST = sorted array. This is THE most important BST property.
- Preorder first element = root. Postorder last element = root. Inorder = root in middle.
- To reconstruct a tree: you need preorder + inorder OR postorder + inorder (not just one).
- All traversals are O(n) time — you must visit every node exactly once.
- Traversal space = O(h): balanced O(log n), skewed O(n) (recursion stack depth).
- BST worst case is ALWAYS when input is sorted → creates a right-skewed tree.
- Use AVL or Red-Black trees in production to prevent worst case O(n).

The 3 Interview Questions That Are The Same:

Q1: Print BST elements in increasing order → INORDER traversal

Q2: Check if a tree is a valid BST → use inorder, check if result is sorted

Q3: Inorder traversal of BST → Left → Root → Right

All 3 answers = the same code. Know this and you get all 3 for free!

Master the recursion. Master the tree.

Every BST operation is just recursion + the BST rule applied at each node.

Understand the recursion trace, and no BST interview question can surprise you.